



1. Что мы сегодня получим

Сейчас я продолжу работу с модулем `testing_lab` в уже открытом учебном проекте.

После предыдущей практики у меня уже есть production-код, контракт и тесты. Теперь я хочу на основании этих файлов создать документацию.

В результате в модуле должны появиться два новых файла:

```
testing_lab\README.md
testing_lab\API.md
```

`README.md` будет точкой входа для разработчика, который впервые открыл модуль.

`API.md` будет описывать программный интерфейс модуля: функции, классы, методы, параметры, возвращаемые значения, ошибки и основные ограничения.

Здесь слово API не означает обязательно HTTP или REST API. У нашего Python-модуля тоже есть API — это публичные функции и классы, которыми пользуется другой код.

2. Проверяем исходные файлы

Сначала открываю PowerShell в корневом каталоге клонированного проекта и сохраняю его путь:

```
$projectRoot = (Get-Location).Path
```

Все дальнейшие команды выполняю из этого каталога. Здесь находятся `.venv`, `testing_lab` и `tests`.

Проверяю содержимое модуля:

```
Get-ChildItem .\testing_lab
```

Я ожидаю увидеть как минимум:

```
__init__.py
refunds.py
CONTRACT.md
```

Теперь проверяю тесты:

```
Get-ChildItem .\tests\testing_lab
```

Здесь должны находиться созданные на предыдущем занятии unit- и integration-тесты.

Проверяю Git:

```
git status --short
```

Перед генерацией документации проверяю состояние целевых файлов. Несвязанные локальные материалы не удаляю и не добавляю в commit.

3. Сначала проверяем, что сам модуль действительно работает

Документировать сломанное состояние проекта не имеет смысла.

Поэтому сначала запускаю тесты:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Я хочу получить успешный результат.

Если тесты сейчас падают, сначала необходимо разобраться с этим состоянием. README не должен сообщать новому разработчику, что модуль готов к работе, если его текущие проверки не проходят.

4. Открываем новую сессию Hermes

Теперь открываю Hermes и создаю новый Chat.

Рабочим каталогом должен быть сохранённый ранее корень проекта.

Для проверки могу использовать:

```
/status
```

После этого смотрю доступные инструменты:

```
/tools list
```

Мне нужны файловые инструменты чтения, поиска и записи, а для проверки команд также terminal.

Если необходимых инструментов нет, проверяю доступные способы их включения в установленной версии Hermes. Названия элементов интерфейса могут различаться между версиями.

Для этой практики мне не требуется отдельный Skill. Я специально хочу показать сам процесс работы агента с исходниками и документацией.

5. Первый промпт: только собираем факты

Я не начинаю с команды:

Напиши README.

Сначала заставляю Hermes разобраться, что вообще находится в модуле.

Отправляю первый рабочий промпт:

Работаем с модулем `testing_lab` в текущем проекте.

Не создавай и не изменяй пока никаких файлов.

Исследуй только следующие источники:

```
testing_lab/refunds.py
testing_lab/CONTRACT.md
tests/testing_lab/conftest.py
tests/testing_lab/test_refunds_unit.py
tests/testing_lab/test_refunds_integration.py
```

Если какого-либо указанного файла нет, сообщи об этом и не придумывай его содержимое.

Считай источники имеющими следующий приоритет:

1. `CONTRACT.md` определяет требуемое бизнес-поведение.
2. `refunds.py` определяет фактический публичный программный интерфейс.
3. Тесты подтверждают реально проверяемые сценарии.

Определи:

назначение модуля;

публичные функции;

публичные классы и методы;

параметры и возвращаемые значения;

исключения, которые явно следуют из реализации;

основные бизнес-ограничения;

зависимости модуля;

как запускаются его тесты;

какие утверждения можно безопасно включить в README;

какие утверждения можно безопасно включить в API-документацию.

Не придумывай отсутствующие возможности.

Если какая-либо информация не подтверждается файлами проекта, пометь её как неизвестную.

После анализа остановись.

Этот промпт я действительно использую перед генерацией.

6. Проверяем анализ Hermes

Теперь я читаю результат.

Я ожидаю, что Hermes обнаружит функцию:

```
calculate_refund(...)
```

класс:

```
ReturnRepository
```

и класс:

```
RefundService
```

Также агент должен увидеть методы:

```
ReturnRepository.create(...)  
ReturnRepository.get(...)  
RefundService.create_request(...)
```

Отдельно я проверяю, не придумал ли он чего-либо лишнего.

Например, в модуле нет HTTP endpoint:

```
POST /refunds
```

Следовательно, в документации такой endpoint появляться не должен.

У нас также нет Redis, Docker, Celery или внешнего платёжного API. Если Hermes внезапно начал их описывать, значит документация уже содержит галлюцинацию.

7. Объясняем, какой README мне нужен

Теперь я отдельно формулирую назначение README.

Я не хочу получить длинное описание каждой функции. Для этого будет `API.md`.

README должен ответить новому разработчику на практические вопросы: что это за модуль, какие задачи он решает, из каких основных компонентов состоит, какие требования важны и как запустить проверки.

Поэтому следующим запросом я задаю уже конкретную структуру результата.

8. Второй промпт: генерируем README

Отправляю Hermes:

На основании выполненного анализа создай:

`testing_lab/README.md`

Это README именно для модуля `testing_lab`, а не для всего репозитория.

Используй только факты из:

`testing_lab/refunds.py`
`testing_lab/CONTRACT.md`
`tests/testing_lab`

Структура README должна содержать:

название и назначение модуля;

короткое объяснение основного сценария;

описание `calculate_refund`;

описание `ReturnRepository`;

описание `RefundService`;

ключевые бизнес-ограничения возврата;

структуру файлов модуля;

минимальный пример использования;

команду запуска тестов;

команду запуска `branch coverage`;

ссылку на API.md;

ссылку на CONTRACT.md.

Для команд используй текущее Windows-окружение проекта.

Команда запуска тестов:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Команды проверки coverage:

```
.\.venv\Scripts\python.exe -m coverage erase
```

```
.\.venv\Scripts\python.exe -m coverage run --branch --source=testing_lab -m pytest tests\testing_lab
```

```
.\.venv\Scripts\python.exe -m coverage report -m
```

Не добавляй:

не существующие зависимости;

не существующие HTTP API;

не существующие переменные окружения;

Docker-инструкции;

процедуру установки проекта, если она не подтверждается текущими файлами;

архитектуру, которой нет в модуле.

Не копируй CONTRACT.md целиком.

Перескажи только правила, необходимые разработчику для понимания модуля.

Все примеры Python должны соответствовать фактическим сигнатурам refunds.py.

Создай файл, но не изменяй production-код, CONTRACT.md и тесты.

После создания сообщи путь созданного файла.

Теперь разрешение на запись распространяется только на:

```
testing_lab\README.md
```

9. Открываем сгенерированный README

После ответа Hermes я не ограничиваюсь его сообщением.

В PowerShell выполняю:

```
Get-Content .\testing_lab\README.md
```

Либо открываю файл непосредственно в редакторе.

Я проверяю, есть ли в README реальные названия компонентов.

Особенно смотрю на пример использования.

Если там вызывается:

```
RefundService.create_refund(...)
```

а реальный метод называется:

```
create_request(...)
```

это уже ошибка документации.

10. Третий промпт: генерируем API.md

Теперь создаю отдельную техническую документацию.

Отправляю следующий реальный промпт:

Теперь создай:

testing_lab/API.md

Документируй публичный Python API файла:

testing_lab/refunds.py

Источником сигнатур является только фактический refunds.py.

Источником бизнес-ограничений является testing_lab/CONTRACT.md.

Тесты используй только как дополнительное подтверждение поведения.

Документируй:

```
calculate_refund;
```

```
ReturnRepository;
```

```
ReturnRepository.__init__;
```

```
ReturnRepository.create;
```

```
ReturnRepository.get;
```

```
RefundService;
```

```
RefundService.__init__;
```

```
RefundService.create_request.
```

Для каждой публичной функции или метода укажи:

фактическую Python-сигнатуру;

назначение;

параметры;

возвращаемое значение;

явно подтверждённые исключения;

существенные ограничения;

небольшой пример использования там, где он действительно полезен.

Не описывай внутренние локальные переменные.

Не придумывай исключения, которых нельзя подтвердить кодом или контрактом.

Не придумывай HTTP endpoints.

Не называй SQLite обязательной production-базой всего приложения. В текущем модуле sqlite3 относится к реализации ReturnRepository в учебном примере.

Отдельно добавь раздел "Поведенческие гарантии", где кратко зафиксируй подтверждённые правила из CONTRACT.md.

Добавь раздел "Границы ответственности", в котором объясни разницу между:

```
calculate_refund;
```

```
RefundService;
```

```
ReturnRepository.
```

Не изменяй никакие существующие файлы.

Создай только `testing_lab/API.md`.

Теперь Hermes должен сформировать именно reference-документацию.

11. Проверяем сигнатуры API

После генерации я открываю:

```
Get-Content .\testing_lab\API.md
```

Теперь открываю рядом:

```
Get-Content .\testing_lab\refunds.py
```

Сравниваю сигнатуры.

Например, в документации для `calculate_refund()` я должен увидеть фактическую форму:

```
calculate_refund(  
    items: list[dict],  
    requested: dict[int, int],  
) -> Decimal
```

Если модель изменила имя параметра или добавила отсутствующий аргумент, я не оставляю это как «примерный вариант».

API reference должен совпадать с кодом.

12. Проверяем описание ReturnRepository

Я отдельно смотрю описание:

```
ReturnRepository
```

Документация должна сообщить, что объект получает:

```
sqlite3.Connection
```

и предоставляет операции создания и получения заявки.

При этом я не хочу видеть утверждение:

ReturnRepository является основной production-БД приложения.

Код этого не подтверждает.

Мы документируем то, что знаем, а не расширяем контекст предположениями.

13. Проверяем описание RefundService

Теперь нахожу:

```
RefundService.create_request
```

Здесь особенно важно корректно передать бизнес-условия.

В документации должно быть понятно, что заказ должен иметь статус:

```
delivered
```

что требуется:

```
delivered_at
```

и что допустимый период включает ровно четырнадцатый день после получения.

Это правило мы специально исправляли на предыдущей практике, поэтому документация должна отражать актуальное состояние.

14. Четвёртый промпт: автоматический аудит документации

Теперь я заставляю Hermes проверить собственный результат.

Отправляю:

Проведи технический аудит созданной документации:

```
testing_lab/README.md
testing_lab/API.md
```

Пока ничего не изменяй.

Сверь каждое техническое утверждение с:

```
testing_lab/refunds.py
testing_lab/CONTRACT.md
```

tests/testing_lab

Особенно проверь:

имена функций и классов;

сигнатуры;

имена параметров;

возвращаемые значения;

описанные исключения;

правило 14 дней;

работу скидки;

начальный статус заявки;

поведение ReturnRepository;

пути к файлам;

внутренние Markdown-ссылки;

Python-примеры;

команды pytest;

команды coverage.

Найди:

утверждения, которых нет в исходниках;

устаревшие утверждения;

противоречия коду;

противоречия CONTRACT.md;

команды, которые ещё не были фактически проверены.

Не исправляй файлы.

Верни отчёт о найденных проблемах.

Это важный этап.

Я использую AI не только как автора документации, но и как её первого reviewer.

15. Пятый промпт: проверяем команды фактическим запуском

Теперь отдельно проверяю исполняемые инструкции.

Отправляю:

Теперь проверь команды, приведённые в testing_lab/README.md.

Не изменяй production-код и тесты.

Фактически выполни в текущем проекте:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

затем:

```
.\.venv\Scripts\python.exe -m coverage erase
```

```
.\.venv\Scripts\python.exe -m coverage run --branch --source=testing_lab -m pytest tests\testing_lab
```

```
.\.venv\Scripts\python.exe -m coverage report -m
```

Не делай вывод об успешности команды до её реального выполнения.

Сравни фактически выполненные команды с текстом README.

Сообщи:

какие команды выполнились;

какие завершились ошибкой;

соответствует ли README реальному окружению проекта.

Ничего пока не исправляй.

Теперь фраза в документации:

Для запуска тестов выполните...

подтверждается реальным запуском.

16. Самостоятельно повторяем проверку

После Hermes я самостоятельно выполняю:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Затем:

```
.\.venv\Scripts\python.exe -m coverage erase
```

```
.\.venv\Scripts\python.exe -m coverage run --branch --source=testing_lab -m pytest tests\testing_lab
```

```
.\.venv\Scripts\python.exe -m coverage report -m
```

Если команды из README совпадают с реально работающими командами, этот раздел документации прошёл проверку.

17. Проверяем ссылки README

Теперь открываю:

```
testing_lab\README.md
```

В нём должны использоваться относительные ссылки примерно такого вида:

```
[API reference](API.md)
```

и:

```
[Контракт модуля](CONTRACT.md)
```

Такие ссылки удобны тем, что при просмотре `testing_lab/README.md` на GitHub они ведут к файлам того же каталога.

Мне не нужна жёстко записанная ссылка на конкретный локальный путь:

```
<локальный абсолютный путь>\...
```

внутри README.

Локальный абсолютный путь относится к нашей машине, а документация репозитория должна оставаться переносимой.

18. Шестой промпт: исправляем только документацию

Если предыдущий аудит нашёл ошибки, теперь разрешаю их исправить.

Отправляю:

Исправь только подтверждённые проблемы документации, найденные при предыдущем аудите.

Разрешено изменять только:

testing_lab/README.md
testing_lab/API.md

Запрещено изменять:

testing_lab/refunds.py
testing_lab/CONTRACT.md
tests/testing_lab

Правила:

сигнатуры должны дословно соответствовать текущему Python API;

бизнес-правила должны соответствовать CONTRACT.md;

команды должны соответствовать тем, которые мы фактически успешно выполнили;

не добавляй предположений;

не расширяй документацию новыми возможностями только ради объёма.

После исправления перечисли, какие именно документальные неточности были устранены.

Обратите внимание, я не даю команду:

Сделай документацию лучше.

Я разрешаю исправить только уже подтверждённые проблемы.

19. Проверяем изменения через Git

Теперь в PowerShell выполняю:

```
git status --short -- testing_lab\README.md testing_lab\API.md
```

В выводе по целевым путям ожидаю увидеть только документацию:

```
testing_lab/README.md
testing_lab/API.md
```

Смотрю diff:

```
git diff -- testing_lab\README.md testing_lab\API.md
```

Так мне удобно увидеть именно то, что создал Hermes.

Если в `git status` появились изменения `refunds.py` или тестов, я отдельно выясняю причину. В этой практике агенту не требовалось их изменять.

20. Проверяем README глазами нового разработчика

Теперь я временно представляю, что впервые вижу этот модуль.

Открываю только:

```
testing_lab\README.md
```

После чтения мне должно быть понятно, зачем существует модуль, какие основные компоненты в нём находятся, что является важным бизнес-ограничением и как запустить тестирование.

Но README не должен заставлять меня читать двадцать экранов reference-информации.

Если мне нужна точная сигнатура метода, я перехожу в:

```
API.md
```

Так я разделяю обзорную и справочную документацию.

21. Проверяем API.md глазами разработчика

Теперь открываю:

```
testing_lab\API.md
```

Допустим, мне необходимо понять:

```
RefundService.create_request(...)
```

Я должен быстро найти его сигнатуру, назначение, параметры, результат и существенные ошибки.

Мне не нужно читать весь README или изучать внутреннюю реализацию.

Такой документ уже действительно выполняет роль API reference.

22. Финальный промпт: документация как проверяемый артефакт

Теперь отправляю последний промпт:

```
Проведи финальную проверку документации модуля testing_lab.

Ничего не изменяй.

Проверь:

testing_lab/README.md
testing_lab/API.md

по отношению к:

testing_lab/refunds.py
testing_lab/CONTRACT.md
tests/testing_lab

Фактически запусти тестовую команду из README.

После этого подготовь краткий отчёт.

Укажи:

какие документы созданы;

какие исходные файлы использовались как источники;

соответствуют ли API-сигнатуры текущему коду;

соответствуют ли бизнес-правила CONTRACT.md;

прошла ли команда pytest;

проверены ли команды coverage;

остались ли утверждения, которые невозможно подтвердить источниками.

Не утверждай успешность никаких проверок, которые фактически не выполнялись.
```

Этот промпт завершает нашу генерацию.

23. Фиксируем документацию

Если всё проверено, выполняю:

```
git add testing_lab\README.md testing_lab\API.md
```

Проверяю staged diff:

```
git diff --staged
```

После этого создаю commit:

```
git commit -m "Практика 3.7: добавить документацию модуля возвратов"
```

Теперь документация хранится рядом с кодом и версионруется вместе с проектом.

24. Что именно мы автоматизировали

В этой работе Hermes не получил задачу просто «написать красивый README».

Сначала агент провёл исследование репозитория с помощью доступных в текущей сессии файловых и терминальных инструментов.

После анализа Hermes сформировал два разных документа.

README.md отвечает за знакомство с модулем и начало работы.

API.md отвечает за точное описание программного интерфейса.

После генерации тот же агент перешёл в другую роль и проверил документацию относительно исходников.

Затем мы отдельно заставили его выполнить команды, которые он указал в README.

И только после этого документацию можно считать не просто сгенерированным текстом, а проверенным артефактом проекта.

25. Итог практики

В результате у нас появляется такая часть проекта:

```
testing_lab/  
  __init__.py  
  refunds.py  
  CONTRACT.md
```

`README.md``API.md`

При этом источники информации остаются разделёнными.

`refunds.py` показывает фактический программный интерфейс.

`CONTRACT.md` определяет требуемое бизнес-поведение.

Тесты показывают, какие сценарии действительно проверяются.

`README.md` помогает новому разработчику начать работу.

`API.md` позволяет использовать публичные функции и классы без необходимости каждый раз разбирать внутреннюю реализацию.

Главный результат этой практики заключается не просто в двух Markdown-файлах. Мы построили воспроизводимую схему генерации документации: сначала исследовать реальные источники, затем создать документацию с ограничением на домыслы, после этого перепроверить её по исходникам и фактически выполнить команды, которые предлагаются пользователю.
